

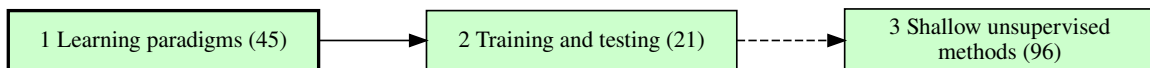
Machine Learning Basics

— Lecture Notes —

Laurenz Wiskott
Institut für Neuroinformatik
Fakultät für Informatik
Ruhr-Universität Bochum, Germany, EU

1 May 2021

— Summary —



1 Learning paradigms: Machine learning is the science of training models on data in order to make good predictions on new data. Depending on the type of data used for training one distinguishes three learning paradigms. If the training data consists of labeled data, i.e. data for which the correct prediction is given, one speaks of *supervised learning*. Examples are classification and regression. If no labels are given and the model is supposed to find structure in the data by itself, one speaks of *unsupervised learning*. Examples are clustering, feature extraction, and dimensionality reduction. *Reinforcement learning* is special in that it assumes an agent that can take actions in an environment and thereby decide itself what data it gets. It receives reward and punishment for its actions from the environment and its goal is to maximize reward and minimize punishment.

Contents

1 Learning paradigms	2
1.1 What is machine learning?	2
1.2 Supervised Learning	4
1.3 Unsupervised Learning	5
1.4 Reinforcement Learning	6
1.5 How to choose the right method? +	8

© 2021 Laurenz Wiskott (ORCID <https://orcid.org/0000-0001-6237-740X>, homepage <https://www.ini.rub.de/PEOPLE/wiskott/>). This work (except for all figures from other sources, if present) is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License, see <https://creativecommons.org/licenses/by-sa/4.0/>. If figures are not included for copyright reasons, they are uni colored, but the word 'Figure', 'Image', or the like in the reference is often linked to a freely available copy.

If applicable, dark red marks core text and sometimes important formulas, a filled diamond ♦ marks important formulas, and an empty diamond ◇ marks less important formulas or items that I would usually also present in a lecture to give context or dive into the details; core text and important formulas are worth remembering and learning for an exam; + marks sections that I would usually skip in a lecture.

You can also download the [teaching material of this topic](#) as zip files and then view them locally on your computer.

1 Learning paradigms

Learning objectives: The learning objective of this unit is that the student can

1. describe what machine learning is
2. define, give an example of, and decide on simple examples whether it is
 - (a) supervised learning
 - i. regression
 - ii. classification
 - (b) unsupervised learning
 - i. clustering
 - ii. blind source separation (cocktail party problem)
 - (c) reinforcement learning
 - i. state, action, reward
 - ii. policy
 - iii. value function
3. intuitively describe the exploration-exploitation tradeoff/dilemma

1.1 What is machine learning?

Learning material:¹

- 8 min Short introduction to machine learning and its three main learning paradigms:
 - 8 min video **What is Machine Learning?**² by Luv Aggarwal at IBM Technology on YouTube
 - 1 0:17 : differences between machine learning, AI, and deep learning
 - 2a 1:32 : supervised learning
 - 2b 4:19 : unsupervised learning
 - 2c 6:38 : reinforcement learning8:22 : end
 - 7 min video **Machine Learning Basics**³ by Simplilearn on YouTube
 - 1 0:00 : what is machine learning?
 - 2a 2:50 : supervised learning
 - 2b 3:45 : unsupervised learning
 - 2c 4:37 : reinforcement learning⁴
 - 1 6:25 : application examples7:24 : end
 - 8 min / 1200 words blog **Introduction to Machine Learning for Beginners**⁵ by Ayush Pant

¹Generic instruction: Consider the (possibly nested) list of resources like a horizontal tree with an invisible root on the very left, and decide from left to right what you want to select to work through. The invisible root node has to be selected. For any selected parent node all children nodes marked with ■ or ● are mandatory and have to be selected. Children nodes marked with □ or ○ are optional and may be selected in addition to get a better understanding of

the material. If a parent node has no mandatory child, then at least one optional child has to be selected. Children marked with + provide additional voluntary material that can be safely ignored, typically going beyond the scope of the section. Children of non-selected parents may be ignored. ■ and □ indicate children that cover (almost) the whole material of the section. Missing content might then be indicated by struck through references to the corresponding learning objectives. ● and ○ indicate children that do not cover (almost) the whole material of the section. They must be combined with others to be complete. Items tend to be ordered by precedence and/or recommended temporal order from top to bottom, assuming that you prefer to first watch a video before reading through lecture notes. If a detailed table of content for videos or lecture notes is given, references to learning objectives might be provided in green, 1:30 should be read as 1 min and 30 seconds, and 1'30 should be read as page 1 at about 30% of the page. Video times may be linked directly to the indicated position in the video, but be aware that the video might be downloaded anew each time you click on a time. Resources without author name are usually authored by Laurenz Wiskott and his team.

²<https://www.youtube.com/watch?v=9gGnTQTYNaE>, accessed 2021-10-05

³<https://www.youtube.com/watch?v=ukzFI9rgwfU>, accessed 2021-10-05

⁴This part is a bit misleading, since there is no real interaction with an environment.

⁵<https://medium.com/data-science/introduction-to-machine-learning-for-beginners-ee6024fdb08>, accessed 2022-11-03

Artificial Intelligence (AI) refers to machines that behave 'intelligently'. The field of AI can broadly be divided into two sub-fields, classical AI and Machine Learning (ML). Classical AI is usually rule based and works on symbolic data, while ML is more a statistical method that is heavily based on learning from data and it usually applies to sub-symbolic data. ML in turn can be divided into more traditional shallow methods and the more modern Deep Learning (DL) with Deep Neural Networks (DNNs).

Shallow methods are largely statistical methods with efficient closed form or iterative solutions, they are well understood analytically, and the results are often easy to interpret and communicate. Shallow methods are usually also quite data efficient.

DNNs is a biologically inspired class of models of enormous complexity, imitating the information processing in the brain, which is carried out by neurons, organized into areas that communicate with each. In NNs the areas are called layers, which often contain thousands of neurons. Older NN have something like 5 layers while modern NNs can have hundreds of layers, which motivates the name 'deep' NN. They usually need huge amounts of data and compute resources for training, and they are hard to interpret. But given enough data and compute they reach extraordinary performance, on specific tasks often super-human. Large Language Models (LLMs) are an example that is quite prevalent in everyday life.

Orthogonal to the shallow vs. deep distinction, there are three major learning paradigms in ML: supervised, unsupervised, and reinforcement learning.

In supervised learning (SL), the model is given input and labels for training. The labels tell the system exactly how to respond to the input. Thus the machine simply learns to reproduce the given label for a given input, at least as well as possible. If the label is categorical, such as an animal species, then the task is classification; if the label is continuous, such as a predicted consumption of electric energy, then the task is regression.

In unsupervised learning (UL) only input data is given, and the task is to discover structure in the data, such as clusters, dimensions, or manifolds. Grouping data into distinct clusters of data points with similar properties might be one of the most natural unsupervised learning tasks that we do all the time automatically. Other tasks are dimensionality reduction and feature extraction. The big advantage of UL is that no labels are needed. This also implies that UL is not specialized to a certain task. Finding structure in data is potentially useful for many tasks.

Reinforcement learning (RL) finally is quite different from the other two in that there is an agent that takes actions to perform a task. Thus, the training data is not simply given to a model, but it is generated by it through actions that it takes. This is the reason why the machine is usually called agent. With respect to the teaching signal provided, RL is somewhere in between SL and UL, since it receives feedback in terms of a scalar reward, i.e. the agent gets told how well it performs but not exactly what it should have done,

so it is more feedback than in UL but less than in SL. The agent can learn by trying random actions in certain situations and seeing how much reward it can get for that. It is surprising how such an agent can learn complicated tasks such as playing back-gammon at world championship level with such simple trial and error strategy.

Let us now consider these three types of learning more closely and define them in terms of the input and output data they use.

1.2 Supervised Learning

Learning material:

- 12 min video [Supervised Learning](#)⁶ by Andrew Ng on RUB server
 - 2(a)i 00:00 : example of a supervised learning problem - regression
 - 2(a)ii 02:59 : example of a supervised learning problem - classification
 - 2a 09:44 : summary and questions - use cases of supervised machine learning
 - 12:17 : end
- 5 min / 900 words blog [A Brief Introduction to Supervised Learning](#)⁷ by Aidan Wilson (you need to register to read, but it'S free)

⁶https://www.ini.rub.de/PEOPLE/wiskott/Teaching/UnsupervisedLearning/StudentsWeb-skj8378df/Extra/Lecture_1.2-Supervised_Learning-Machine_Learning_Andrew_Ng-lowRes360.mp4, accessed 2024-10-04

⁷<https://medium.com/data-science/a-brief-introduction-to-supervised-learning-54a3e3932590>, accessed 2022-11-03

In SL the training data consists of pairs of input $\mathbf{x}^\mu$ and output $\mathbf{y}^\mu$, where μ is an index for the different data pairs. The machine usually does not generate the output immediately and possibly never exactly. Thus it is useful to quantify the mismatch between required output $\mathbf{y}^\mu$ and actual output $\hat{\mathbf{y}}^\mu$ with a so called loss, error, or cost function $L(\mathbf{y}, \hat{\mathbf{y}})$. During training, the parameters of the machine are modified such that this loss function is minimized on the training data, and then the hope is that the machine also performs well on new data.

Regression Assume the input consists of the current electric energy consumption in a community and the weather forecast for the following day and the output consists of the energy consumption on the following day. Then a machine can be trained on historic data to be able to then predict the energy consumption on the next day and set a suitable price for electricity. Since the output is a continuous quantity, the task is a regression task. A very common loss function for such training could be the mean squared error (MSE) loss:

$$L(\mathbf{y}, \hat{\mathbf{y}}) = \|\mathbf{y} - \hat{\mathbf{y}}\|^2 \quad (1.1)$$

This loss needs to be minimized and simply punishes quadratically any deviation from the desired value.

Classification Now assume the input is an image and the output a class membership, such as 'dog', 'bus', 'cat', or 'person'. Then a machine can be trained on a large set of hand-labeled images and then be used to do automated image classification. Since the output is categorical, the task is a classification task. In such cases the output is usually represented by a one-hot encoded vector rather than a simple number indicating the class. With the four classes from above in that order, (1, 0, 0, 0) would represent a 'dog', (0, 1, 0, 0) would represent a 'bus', etc. One-hot encoding has the big advantage that it can be generalized to express uncertainty with a vector of non-negative numbers between 0 and 1 normalized to sum one, e.g. (0.7, 0, 0.3, 0)

could indicate a 70% probability for 'dog' and a 30% probability for 'cat'. This is particularly helpful in the initial phase of training, when the machine is not very good at classification yet. A most common loss function for such a representation is the cross-entropy loss:

$$L(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_i y_i \log(\hat{y}_i) \quad (1.2)$$

with $0 \log(0) := \lim_{x \rightarrow 0} x \log(x) = 0$. Remember that $\mathbf{y}$ is the one-hot encoded true class label and $\hat{\mathbf{y}}$ is a vector of predicted probabilities for the different classes. Since all but one entry of $\mathbf{y}$ is zero, the loss function reduces to

$$L(\mathbf{y}, \hat{\mathbf{y}}) = -\log(\hat{y}_{i^*}) \quad (1.3)$$

if the correct class index is i^* . Minimizing the loss function means maximizing $\hat{y}_{i^*}$. One advantage of this loss function over the MSE loss is that it penalizes confident misclassifications very strongly, so that it is more important to increase the probability for the correct class from 0.1 to 0.2 for one image than from 0.8 to 0.9 for another image.

Most ML applications nowadays is based on SL. One limitation, however, is that it might be expensive to generate good training data, because they need to be labeled by an expert. One therefore often takes labels that are easy to get, if that helps, such as one part of an image given the other, or one word given the preceding words. If one defines input $\mathbf{x}$ to be the words up to some point and output $\mathbf{y}$ the next word, it is trivial to generate a lot of data without the need for expert labeling, and the resulting machine can produce text by simple next word prediction. This is how simple LLMs work.

1.3 Unsupervised Learning

Learning material:

- 11 min video **Unsupervised Learning**⁸ by Andrew Ng on RUB server
 - 2a 00:06 : supervised learning (reminder)
 - 2b 00:28 : unsupervised learning
 - 2(b)i 00:50 : examples of an unsupervised learning problem - clustering
 - 2(b)ii 05:23 : examples of an unsupervised learning problem - cocktail party problem
 - 09:18 : ~~programming in octave~~
 - 2b 12:16 : questions - use cases of unsupervised machine learning
 - 13:57 : end

⁸https://www.ini.rub.de/PEOPLE/wiskott/Teaching/UnsupervisedLearning/StudentsWeb-skj8378df/Extra/Lecture_1.3-Unsupervised_Learning-Machine_Learning_Andrew_Ng-lowRes360.mp4, accessed 2024-10-04

UL is based on just input data $\mathbf{x}$ and a somewhat vague goal of finding structure in the data that could be useful for downstream tasks, such as regression, classification, storage, or visualization. So, UL can be used as pre-processing for SL, e.g. if there is simply too much data to handle or too little labeled data to do SL directly. While there are basically only two types of tasks in SL, namely regression and classification, there are many more types in UL. Two common ones are clustering and dimensionality reduction.

Clustering Clustering means that you find groups of data points that are close to each other and distinct from other clusters. One way to represent a single cluster is by a prototype, i.e. a typical example representative of all the other members of that cluster. So, if the data samples are given as $\mathbf{x}^\mu$, the prototypes would

be given by $\mathbf{c}^\alpha$ in the same vector space. Each data point can then be assigned to one cluster, and it is quite reasonable to assign a data point to the prototype that is nearest to it. One could write for instance:

$$\alpha^*(\mathbf{x}^\mu) = \max_{\alpha} \|\mathbf{c}^\alpha - \mathbf{x}^\mu\|. \quad (1.4)$$

It is also reasonable to assume that the prototype should be close to all its assigned data points, so that one chooses it such that the MSE loss

$$L(\mathbf{c}^\alpha, \mathbf{x}^\mu) = \sum_{\mu \in \mathcal{S}_\alpha} \|\mathbf{c}^\alpha - \mathbf{x}^\mu\|^2 \quad (1.5)$$

is minimal, where $\mathcal{S}_\alpha$ is the set of data points assigned to cluster α . Finally, also the number of clusters needs to be determined, which can be done by some measure of cluster separation or improvement of the MSE loss if you add a new cluster. This example shows that UL is conceptually less straight forward than SL.

Dimensionality Reduction The task in (linear) dimensionality reduction is to linearly reduce the dimensionality of the data while preserving as much information as possible. Reducing dimensionality can be done by an $M \times N$ matrix $\mathbf{V}$ with $M < N$, if N is the original dimensionality of the data. We also need an $N \times M$ matrix that recovers the original data as well as possible, we call it $\mathbf{W}$. Information loss can be formalized by the MSE loss again, so that we are looking for a pair of matrices $\mathbf{V}$ and $\mathbf{W}$ that minimizes

$$L(\mathbf{V}, \mathbf{W}) = \sum_{\mu} \|\mathbf{x}^\mu - \mathbf{WVx}^\mu\|^2. \quad (1.6)$$

Other Types of UL Other types of UL include separating mixtures of signals, extracting features, selecting features, estimating a probability distribution, nonlinear compression, nonlinear flattening for visualization and many more.

1.4 Reinforcement Learning

Learning material:

- 14 min video **Reinforcement Learning for Engineers, Part 1: What Is Reinforcement Learning?**⁹ by Brian at MATLAB on YouTube
 - 2c 00:00 : Introduction
 - 2c 01:37 : Example: controlling a robot
 - 2b 02:51 : Unsupervised learning
 - 2a 03:32 : Supervised learning
 - 2c 05:05 : Reinforcement Learning (RL)
 - 2(c)i 05:50 : State, action, reward
 - 2(c)ii 06:36 : Learning a policy
 - 2(c)iii 08:15 : Value function
 - 3 09:56 : Exploration/exploitation tradeoff
 - 2c 11:13 : RL solves a control problem
 - 2c 12:25 : Prerequisites for RL
 - 13:31 : end

⁹<https://www.youtube.com/watch?v=pc-H4vyg2L4>, accessed 2021-10-06

To formalize reinforcement learning, we need a number of concepts and notions. Let us introduce them by means of a simple example. In reinforcement learning we consider **an agent** (D: Agent), which is

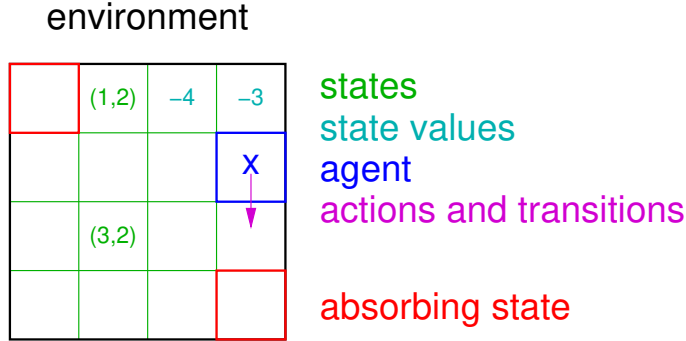


Figure 1.1: A simple example of reinforcement learning to introduce basic notions.

© CC BY-SA 4.0

in one of a number **states** (D: Zustände) s in **an environment** (D: Umgebung). In the simple example shown here the environment is simply a 4×4 field and each field is a state $s \in \{(1, 1), (1, 2), \dots, (4, 4)\}$. The agent can perform certain **actions** (D: Aktionen) a that will usually cause **transitions** (D: Übergänge) into neighboring states s' . Here the actions are movements right, down, left, or up, i.e. $a \in \{\rightarrow, \downarrow, \leftarrow, \uparrow\}$. The transitions may either be deterministic or, more generally, probabilistic with certain unknown **transition probabilities** (D: Übergangswahrscheinlichkeiten). In the former case one might introduce a transition function S and write $s' = S(s, a)$; in the latter case one has to use probabilities and write $P(s'|s, a)$. In our simple example we would have, for instance, $S((2, 3), \uparrow) = (1, 3)$. Upon each action taken in a certain state, the agent receives an **immediate reward** (D: unmittelbare Belohnung) $r = R(s, a)$, which may actually be negative and then is effectively a punishment (D: Bestrafung). In our example the immediate reward is -1 , thus each transition gets punished and the goal of the agent should obviously be to reach one of the two absorbing states as quickly as possible to minimize the overall punishment (maximize the overall reward). An **absorbing state** (D: absorbierender Zustand?) is a state at which the agent stops and the task is over, which may be good or bad. The **expected reward** (D: erwartete Belohnung) is the expectation value of the reward summed over the future given the agent is in state s_0 at time t_0 , i.e.

$$E \left\{ \sum_{t=t_0}^{\infty} r(t) \mid s(t_0) = s_0 \right\}. \quad (1.7)$$

The agent has a **policy** (D: Strategie) π , which assigns an action $a = A(s)$ to a state. If the policy is deterministic, it makes little sense to distinguish $\pi(s)$ and $A(s)$, but if it is probabilistic it does. Finally, the agent also has a **value function** (D: Bewertungsfunktion) $V(s)$ that represents the estimated expected reward for each state. Ideally $V(s) = E\{\sum_{t=t_0}^{\infty} r(t) \mid s(t_0) = s\}$. The value function obviously depends on the policy, see Figure 1.2.

The policy is usually found by a lot of trial and error. One way is to initialize the value function to some optimistic but probably incorrect values and follow a greedy policy and always chose the action with the highest value. As the agent transitions from s to state s' and receives reward r , one can improve the value function in state s . In a deterministic system that would simply be $V(s) = V(s') + r$. As the agent takes more and more actions, the value function gets more and more faithful and can guide the actions better and better leading to a better policy.

One problem with this method is that, if a state has an overly pessimistic value, the agent will always avoid that state and never correct its value. This can be solved by occasionally also taking sub-optimal actions

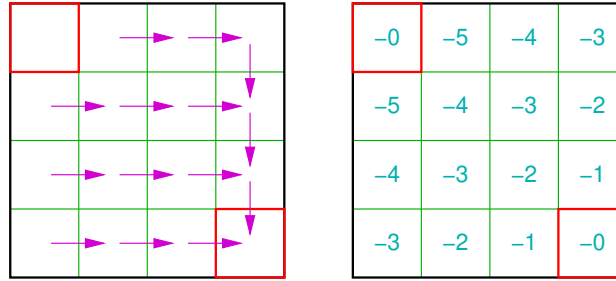


Figure 1.2: A possible policy and the corresponding value function for the simple example of Figure 1.1.

© CC BY-SA 4.0

to explore also unfavorable actions and correct the corresponding value function. The trade-off between choosing optimal actions given the information the agent has and choosing sub-optimal actions to improve the information is referred to as *exploration-exploitation trade-off* or *dilemma*.

The first major achievement of RL was to learn backgammon at world championship level (Tesauro, 1995).

1.5 How to choose the right method? +

Learning material:

- 11 min video [How Data Scientist choose an Algorithm Smartly](#)¹⁰ [1:08-12:37] by The AI University on YouTube
- 5 min overview sheet [Machine Learning Algorithm Cheat Sheet](#)¹¹ at Microsoft

¹⁰<https://www.youtube.com/watch?v=dzH69iV-4MM&t=68>, accessed 2021-11-03

¹¹<https://learn.microsoft.com/en-us/azure/machine-learning/algorithm-cheat-sheet?view=azureml-api-1>, accessed 2022-11-03

References

Tesauro, G. (1995). Temporal difference learning and TD-Gammon. *Commun. ACM*, 38:58–68.